Experiment-9

Aim : To study and implement **DCL (Data Control Language)** commands like **GRANT**, **REVOKE**, and **TCL command SAVEPOINT**.

- ✓ **GRANT** → Gives privileges to users on database objects.
- ✓ **REVOKE** → Removes privileges from users.
- ✓ **SAVEPOINT** → Creates a point in a transaction to which we can rollback later.

Step 1: Create database and use it.

```
CREATE DATABASE Lab_Assignment_9;
▷Run
USE Lab_Assignment_9;  4ms
```

Step 2: Create Tables

```
CREATE TABLE Departments (
    dept_no INT,
    dept_name VARCHAR(50),
    dept_location VARCHAR(50)
);
CREATE TABLE Employees (
    emp_id INT,
    emp_name VARCHAR(50),
    emp_salary DECIMAL(10,2),
    dept_no INT
);  35ms
```

Step 3: Insert Values

```
INSERT INTO Departments VALUES
(101, 'HR', 'Delhi'),
(102, 'Finance', 'Mumbai'),
(103, 'IT', 'Bangalore');
▷Run | Select
INSERT INTO Employees VALUES
(1, 'Jony Gautam', 50000, 101),
(2, 'Varun chauhan', 60000, 102),
(3, 'Ravi', 55000, 103),
(4, 'Pranjal Dutta', 52000, 101),
(5, 'Ankit', 58000, 102);
```

(a) Grant ALL privileges on Employees table

```
CREATE USER 'user1'@'localhost' IDENTIFIED BY 'password123';
▷Run
GRANT ALL PRIVILEGES ON Employees TO 'user1'@'localhost';
▷Run
FLUSH PRIVILEGES;
```

Explanation

- Gives **all permissions** (SELECT, INSERT, UPDATE, DELETE, etc.)
- user1 can fully access the **Employees** table

(b) Grant SOME privileges on Employees table

```
GRANT SELECT, INSERT ON Employees TO 'user1'@'localhost';
```

Explanation

- Gives only:
  - SELECT → read data
  - INSERT → add new records
- User cannot update or delete data

(c) Revoke ALL privileges

```
REVOKE ALL PRIVILEGES ON Employees FROM 'user1'@'localhost';
```

Explanation

- Removes all permissions from user1
- User cannot access the table anymore

(d) Revoke SOME privileges

```
REVOKE INSERT ON Employees FROM 'user1'@'localhost';
```

Explanation

- Removes only INSERT permission
- User can still perform other allowed operations (like SELECT)

(e) Implement SAVEPOINT

```
START TRANSACTION;
▷Run
UPDATE Employees
SET emp_salary = emp_salary + 1000;
▷Run
SAVEPOINT sp1;
▷Run
UPDATE Employees
SET emp_salary = emp_salary + 2000;
▷Run
ROLLBACK TO sp1;
▷Run
COMMIT;
SELECT * FROM Employees;
```

Output:

emp_id int	emp_name varchar	emp_salary decimal	dept_no int
1	Jony Gautam	50000.00	101
2	Varun chauhan	60000.00	102
3	Ravi	55000.00	103
4	Pranjal Dutta	52000.00	101
5	Ankit	58000.00	102

Explanation

1. START TRANSACTION → Begins transaction
2. First UPDATE → Salary increased
3. SAVEPOINT sp1 → Marks a checkpoint
4. Second UPDATE → Another change
5. ROLLBACK TO sp1 → Undo second update only
6. COMMIT → Save final changes permanently